

Matura paper

Using generative AI to create music

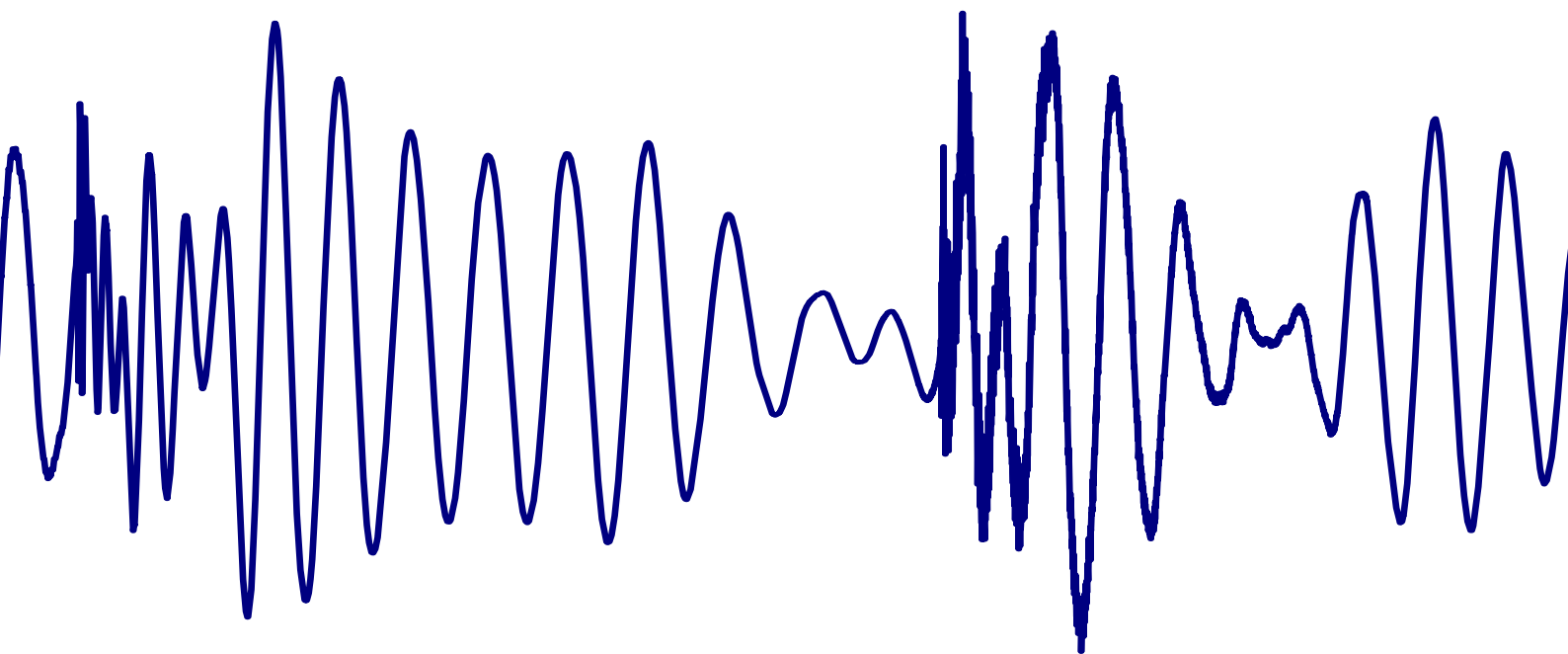
*How can I use generative AI to create music based on DJ sets
as training data?*

Finian Landes 4Ae, Gymnasium Liestal

Supervisor: Robyn Steiner-Curtis

Major subject: Applied Mathematics/Physics

Submission date: 22.10.2025



Motivation

As part of my maturation project work, I wanted to deepen my understanding of artificial intelligence and machine learning, a topic I first encountered in the project work (PA) on supervised learning using PCA and K-means. I am also passionate about music, especially electronic music, so it felt natural for me to combine these two interests. In this project, I will therefore explore how AI can be used to connect my interests in programming and music composition.

Table of Contents

1	INTRODUCTION	5
2	PROBLEM DESCRIPTION.....	5
3	MACHINE LEARNING AND GENERATIVE MODELS.....	6
3.1	DIFFUSION MODEL	6
3.1.1	<i>Denoising Diffusion Probabilistic Models (DDPM)</i>	6
3.1.2	<i>Denoising Diffusion Implicit Models (DDIM)</i>	9
3.1.3	<i>V-objective diffusion</i>	10
3.2	NEURAL NETWORKS.....	12
3.2.1	<i>General neural networks</i>	12
3.2.2	<i>Network conditioning</i>	12
3.2.3	<i>Building blocks</i>	13
3.2.4	<i>Training neural networks</i>	15
3.2.5	<i>U-Net architecture</i>	19
4	AUDIO REPRESENTATION AND PROCESSING.....	20
4.1	WAVEFORM	20
4.2	SPECTROGRAM.....	21
4.2.1	<i>STFT spectrogram</i>	21
4.2.2	<i>Mel spectrogram</i>	22
4.2.3	<i>Audio reconstruction</i>	23
5	ETHICAL CONSIDERATIONS.....	23
6	EXPERIMENT SETUP	24
6.1	TRAINING DATA	24
6.1.1	<i>Preprocessing</i>	24
6.2	TEST DATASET	24
6.3	HARDWARE AND SOFTWARE	24
7	EXPERIMENTS.....	25
7.1	NAIVE DIFFUSION IMPLEMENTATION.....	25
7.2	DIFFUSION BASED ON OTHER PROJECTS.....	26
7.3	AUDIO DIFFUSION PYTORCH	27
7.3.1	<i>Two-dimensional diffusion</i>	27
7.3.2	<i>One-dimensional diffusion</i>	27
7.4	UNCONDITIONAL GENERATION USING THE MOÛSAI ARCHITECTURE.....	28
7.5	VOCODERS.....	29

7.6	VQ-VAE AND TRANSFORMER	30
7.7	VAE	31
7.8	DIRECT WAVE DIFFUSION	32
7.8.1	<i>Network architecture and activations</i>	33
7.8.2	<i>Receptive field and convolutions</i>	33
7.8.3	<i>Results and analysis</i>	34
7.8.4	<i>Limitations and possible improvements</i>	38
8	REFLECTION	39
9	REFERENCES	41
9.1	BIBLIOGRAPHY	41
9.2	TABLE OF FIGURES	43
10	ACKNOWLEDGEMENTS	45
11	APPENDIX	46
11.1	GITHUB REPOSITORY	46
11.2	HUGGING FACE	46

1 Introduction

Artificial intelligence (AI) has become part of our everyday lives, from text generation with large language models to photorealistic image creation, speech synthesis, self-driving cars¹, and even in medicine, such as cancer diagnosis² from MRI scans. Among these, generative AI has made remarkable progress by not only producing high-quality images, text and videos, but also artificial music generation. Music, however, is particularly challenging because it is continuous and high-dimensional with long-term dependencies. While one might not rate music generation as the most important application of AI, its dimensional complexity makes it an interesting domain to test and scale generative AI models.

This project explores whether it is possible to design and train such generative models with limited and accessible resources, and without prior experience in machine learning. In the following, the theoretical foundations of neural networks and diffusion models will be introduced and the experiments will be presented, then the results and limitations will be discussed and finally the work will be reflected on.

2 Problem description

The leading question of this project is: *“How can I use denoising diffusion probabilistic models in deep learning to create music based on DJ sets as training data?”*

Diffusion models were chosen because they represent the current state of the art in generative AI, outperforming alternatives such as generative adversarial networks (GANs) in many areas (Kong, Kim and Bae, 2020; Huang *et al.*, 2022). DJ sets provide a suitable and large dataset, as they consist of continuous music without gaps, making segmentation easy. Additionally, manual annotation or preprocessing of the music samples is wanted, as this would be too tedious and time-consuming for a project of this scale. Since no manual annotations are required, this approach falls into the category of unsupervised learning (“Unsupervised learning,” 2025).

¹ Self-Driving Cars using AI, [online] <https://waymo.com/waymo-driver/> (Accessed: September 22, 2025).

² Aftab, M et al., 2025: AI in Oncology: Transforming Cancer Detection through Machine Learning and Deep Learning Applications, [online] <https://doi.org/10.48550/arXiv.2501.15489>.

3 Machine learning and generative models

3.1 Diffusion model

Diffusion models are a type of generative AI model that can create new data, such as images³ or audio. Unlike other methods, such as GANs or variational autoencoders, which produce new outputs in one step, diffusion models iteratively refine a noisy input until it is clear. This increases computational complexity but enables them to generate high-quality outputs.

Because diffusion was first applied to image data, many explanations and examples in the following theory section refer to images. However, the same principles apply to audio data, with only minor adaptations to handle the one-dimensional time domain.

For training Sohl-Dickstein *et al.* (2015) introduced a method to gradually add noise to the data, steadily corrupting its structure. A neural network is then trained to reverse this process, learning to construct meaningful data from noise. There are several diffusion methods, such as DDPM, DDIM and v-objective diffusion, which will be explored in the following sections. (Sohl-Dickstein *et al.*, 2015; Ho, Jain and Abbeel, 2020)

3.1.1 Denoising Diffusion Probabilistic Models (DDPM)

Denoising diffusion probabilistic models were first introduced in 2015 by Sohl-Dickstein *et al.* and later applied to image synthesis by Ho *et al.* in 2020. The name and idea of diffusion are inspired by diffusion processes in physics, where systems evolve gradually through random perturbations (Sohl-Dickstein *et al.*, 2015). The DDPM algorithm uses a Markov chain to gradually add noise to the data during the so-called forward diffusion process (Ho, Jain and Abbeel, 2020). A neural network, typically a U-Net, is trained to perform the backward diffusion, removing noise from the data. (Ho, Jain and Abbeel, 2020)

This can be seen in Figure 1.

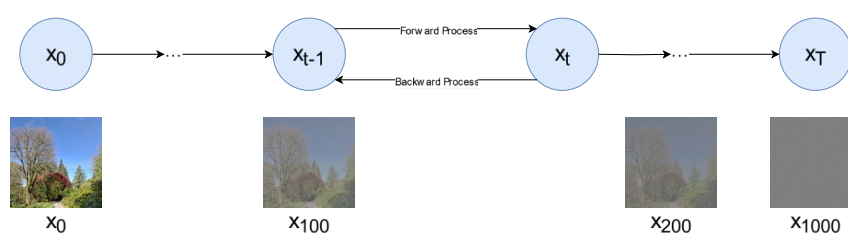


Figure 1: Forward and backward diffusion process

³ Stable Diffusion, [online] <https://stability.ai/>. DALL-E, [online] <https://openai.com/index/dall-e-2/>.

3.1.1.1 The Training Process

For the diffusion process the number of timesteps T (typically 1000) is chosen (Ho, Jain and Abbeel, 2020). It defines how many steps are needed to transition from a clear image x_0 to noise x_T . During the diffusion process, Gaussian noise ϵ is added according to a schedule $\beta_1, \dots, \beta_T$, which determines how much noise is added at each timestep t . There are different noise schedules e.g. a linear schedule where noise is added evenly or a cosine schedule defined as $\alpha_t = \frac{f(t)}{f(0)}, f(t) = \cos^2\left(\frac{t+s}{1+s} \cdot \frac{\pi}{2}\right)$ where s is commonly 0.008. This improves output quality by preserving more information until later steps (Nichol and Dhariwal, 2021). This is shown in Figure 2.

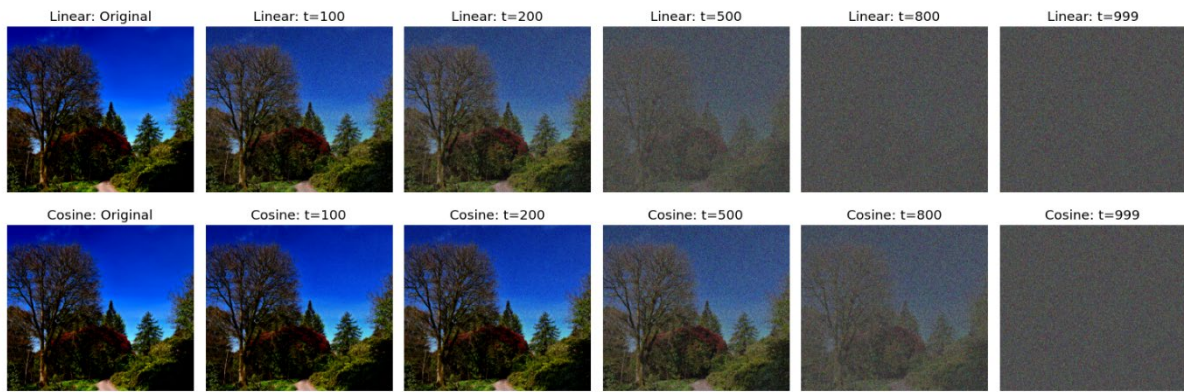


Figure 2: Linear vs. cosine noise schedule, visualized with images for clarity

During training the following steps are performed:

1. An image x_0 is chosen from the training dataset.
2. A timestep t between 1 and T is randomly sampled. Additionally, noise ϵ is generated from a standard normal distribution $\mathcal{N}(0, I)$.
3. The noisy image x_t is then created according to the following formula (Ho, Jain and Abbeel, 2020).

$$\begin{aligned}\alpha_t &= 1 - \beta_t \\ \bar{\alpha}_t &= \prod_{s=1}^t \alpha_s \\ x_t &= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon\end{aligned}$$

x_t and t are then fed to the neural network ϵ_{θ} . t is included as an input to enable the network to learn the amount of noise to remove at each step. Instead of training a model to predict the clear image from the noisy image, it is trained to predict the original noise added to the image, as this improves output quality. This is known as the epsilon prediction trick introduced

by Ho *et al.* (2020). Lastly the mean squared error (MSE) is calculated to perform a gradient descent step. This algorithm can be summarized as shown in Figure 3. (Ho et al., 2020; Sohl-Dickstein et al., 2015)

```

1: repeat
2:    $x_0 \sim q(x_0)$ 
3:    $t \sim \text{Uniform}(\{1, \dots, T\})$ 
4:    $\epsilon \sim \mathcal{N}(0, I)$ 
5:   Take gradient descent step on
        $\nabla_{\theta} \|\epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon, t)\|^2$ 
6: until converged

```

Figure 3: DDPM training algorithm (adapted from Ho et al., 2020)

3.1.1.2 Sampling

For the sampling process, x_T is first sampled from a standard normal distribution $\mathcal{N}(0, I)$ (Ho, Jain and Abbeel, 2020). The following steps are then repeated for $t = T$ down to 1:

1. z is sampled from noise $\mathcal{N}(0, I)$, except for the last step where $z = 0$. This is because there should be no noise on the final image.
2. x_{t-1} is calculated with the following formula, $\epsilon_{\theta}(x_t, t)$ is the network's prediction (Ho, Jain and Abbeel, 2020).

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

The term $\sigma_t z$, where σ_t is $\sqrt{\beta_t}$ or a learned parameter, is added to introduce controlled randomness during sampling so that the sampling process is nondeterministic. (Sohl-Dickstein *et al.*, 2015; Ho, Jain and Abbeel, 2020)

This can also be described in pseudocode shown in Figure 4.

```

1:  $x_t \sim \mathcal{N}(0, I)$ 
2: for  $t = T, \dots, 1$  do
3:   if  $t > 1$  then
4:      $z \sim \mathcal{N}(0, I)$ 
5:   else
6:      $z = 0$ 
7:   end if
8:    $\sigma_t = \sqrt{\beta_t}$ 
9:    $x_{t-1} = \frac{1}{\sqrt{\alpha_t}} (x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t)) + \sigma_t z$ 
10: end for
11: return  $x_0$ 

```

Figure 4: DDPM sampling algorithm (adapted from Ho et al., 2020)

3.1.2 Denoising Diffusion Implicit Models (DDIM)

To speed up inference⁴ Song *et al.* proposed denoising diffusion implicit models in 2020. The DDIM approach moves away from the Markov chain, allowing steps to be skipped during sampling without losing quality and therefore increasing inference speed. Another benefit of DDIM is the possibility to make the diffusion process continuous, which increases the quality of the output. This can be done by taking a very large T during training, which would take longer with DDPM, but for DDIM, this does not increase sampling time as the number of steps used for sampling can be chosen during sampling. (Song, Meng and Ermon, 2020)

3.1.2.1 Training

The training objective is the same as for DDPM, which means that the training from DDPM can be used. Furthermore, this means that the sampling process can be defined after training.

3.1.2.2 Sampling

For sampling, x_t is first sampled from a standard normal distribution $\mathcal{N}(0, I)$. The process then iterates backward from T to 1. To compute x_{t-1} the following formulae are used,

$$\begin{aligned}\sigma_t &= \eta \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}} \\ \hat{\epsilon} &= \epsilon_\theta(x_t, t) \\ \hat{x}_0 &= \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}} \\ x_{t-1} &= \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \hat{\epsilon} + \sigma_t z \quad \text{where } z \sim \mathcal{N}(0, I)\end{aligned}$$

where η controls the stochasticity ($\eta = 0$ meaning the process is fully deterministic). As with DDPM, the last term is added to introduce controlled randomness. (Song, Meng and Ermon, 2020)

⁴ Inference is the process of using a trained neural network to make predictions or generate new data.

This process can be visualized using the pseudocode shown in Figure 5.

```

1:  $\eta \in [0, 1]$ 
2:  $x_t \sim \mathcal{N}(0, I)$ 
3: for  $t = T, \dots, 1$  do
4:   if  $t > 1$  then
5:      $z \sim \mathcal{N}(0, I)$ 
6:   else
7:      $z = 0$ 
8:   end if
9:    $\sigma_t = \eta \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}}$ 
10:   $\hat{\epsilon} = \epsilon_\theta(x_t, t)$ 
11:   $\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}$ 
12:   $x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \hat{\epsilon} + \sigma_t z$ 
13: end for
14: return  $x_0$ 

```

Figure 5: DDIM sampling algorithm

3.1.3 V-objective diffusion

V-objective diffusion is less common than DDPM or DDIM, as it uses a different training objective. It is similar to DDIM, because the model is trained with a continuous timestep schedule. Flavio Schneider proposed using this method for audio diffusion, since it produced the best results compared to DDPM and DDIM. Instead of predicting the original noise, the model is trained to predict a velocity (shown in blue in Figure 6), from which both the noise (red) and the denoised image (yellow) can be inferred. (Schneider, 2023)

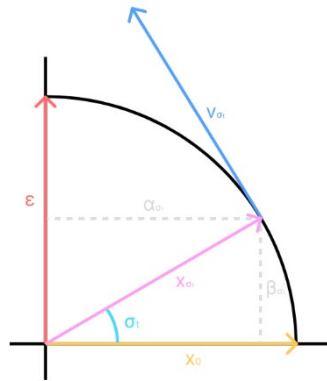


Figure 6: V-Objective diffusion circle
(Schneider, 2023)

3.1.3.1 Training

During training, a continuous noise level $\sigma \in [0, 1]$ is chosen, and ϵ is sampled from a standard normal distribution $\mathcal{N}(0, I)$. The following equations are used to find x_σ and x_0 .

$$\alpha_\sigma = \cos\left(\frac{\pi}{2}\sigma\right), \quad \beta_\sigma = \sin\left(\frac{\pi}{2}\sigma\right)$$

$$x_\sigma = \alpha_\sigma x_0 + \beta_\sigma \epsilon$$

These equations allow the algorithm to move linearly from x_0 to x_1 , where x_1 corresponds to pure noise. Because the timesteps σ are sampled uniformly from $[0,1]$, the model is exposed to more training samples that are closer to x_0 than to pure noise, which is beneficial for training. This can be compared to using a cosine noise schedule for DDPM/DDIM. The model predicts the velocity $\hat{v}_\sigma$, from which x_0 , ϵ_σ and $x_{\sigma_{\text{prev}}}$ can be inferred. This can formally be represented as follows.

$$\begin{aligned}\hat{v}_\sigma &= \epsilon_\theta(x_\sigma, \sigma) \\ x_0 &= \alpha_\sigma x_\sigma - \beta_\sigma \hat{v}_\sigma \\ \epsilon_\sigma &= \beta_\sigma x_\sigma + \alpha_\sigma \hat{v}_\sigma\end{aligned}$$

The previous step value $x_{\sigma_{\text{prev}}}$ can be computed as follows, although it is not used for training.

$$x_{\sigma_{\text{prev}}} = \alpha_{\sigma_{\text{prev}}} x_0 + \beta_{\sigma_{\text{prev}}} \epsilon_\sigma$$

Finally, the training loss, the mean squared error (MSE) is calculated between the predicted velocity $\hat{v}_\sigma$ and the true velocity $v_\sigma = \alpha_\sigma \epsilon_\sigma - \beta_\sigma x_\sigma$. (Schneider, 2023)

The algorithm including the training objective can be seen in Figure 7.

```

1: repeat
2:    $\sigma \sim [0, 1]$ 
3:    $\epsilon \sim \mathcal{N}(0, I)$ 
4:    $\alpha_\sigma = \cos(\frac{\pi}{2}\sigma)$ ,  $\beta_\sigma = \sin(\frac{\pi}{2}\sigma)$ 
5:    $x_\sigma = \alpha_\sigma x_0 + \beta_\sigma \epsilon$ 
6:    $v_\sigma = \alpha_\sigma \epsilon - \beta_\sigma x_\sigma$ 
7:   Take gradient descent step on
        $\nabla_\theta \|\epsilon_\theta(x_\sigma, \sigma) - v_\sigma\|^2$ 
8: until converged

```

Figure 7: V-Objective training algorithm

3.1.3.2 Sampling

For sampling, the noise x_t is drawn from a standard normal distribution $\mathcal{N}(0, I)$. In sampling the continuous noise level σ is replaced by discrete timesteps t starting at T (usually set between 10 and 100). Then the model predicts x_{t-1} , and this process is repeated until $t = 1$ using a linear schedule. (Schneider, 2023)

The v-objective sampling process can be summarized, as shown in Figure 8.

```

1:  $x_t \sim \mathcal{N}(0, I)$ 
2: for  $t = T, \dots, 1$  do
3:    $\alpha_t = \cos(\frac{\pi}{2} \frac{t}{T}), \beta_t = \sin(\frac{\pi}{2} \frac{t}{T})$ 
4:    $\alpha_{t-1} = \cos(\frac{\pi}{2} \frac{t-1}{T}), \beta_{t-1} = \sin(\frac{\pi}{2} \frac{t-1}{T})$ 
5:    $\hat{v} = \epsilon_\theta(x_t, t)$ 
6:    $\hat{x} = \alpha_t x_t - \beta_t \hat{v}$ 
7:    $\hat{\epsilon} = \beta_t x_t + \alpha_t \hat{v}$ 
8:    $x_{t-1} = \alpha_{t-1} \hat{x} + \beta_{t-1} \hat{\epsilon}$ 
9: end for
10: return  $x_0$ 

```

Figure 8: V-Objective sampling algorithm

3.2 Neural networks

3.2.1 General neural networks

The neural network is the part of the project that is capable of learning data distributions. Neural networks (neural nets) are loosely inspired by the human brain in that they are made up of interconnected neurons. Each neuron applies a linear transformation to its input. The weights of these linear transformations are referred to as model parameters and they are learnable. There are different ways to arrange neurons, which changes the way the data is processed. The simplest example is the fully connected (linear) layer. For this project however, convolutional layers are the more relevant building blocks. (“Neural network (machine learning),” 2025)

3.2.2 Network conditioning

For diffusion training the timestep t is passed to the model alongside the input. This is done to allow the model to know how much noise to remove at each timestep. To convert this information into a more useful representation, sinusoidal embeddings are used. Each embedding dimension i alternates between sine and cosine functions with exponentially decreasing wavelengths:

$$p_t^{(i)} = \begin{cases} \sin(\omega_k t), & \text{if } i = 2k \\ \cos(\omega_k t), & \text{if } i = 2k + 1 \end{cases}$$

$$\omega_k = \frac{1}{10000^{\frac{2k}{d}}}$$

Where d is the embedding dimension (128 in most of the experiments) and $p_t^{(i)}$ the embedding vector. (Vaswani *et al.*, 2023)

This can be visualized as shown in Figure 9. Each row in the plot corresponds to the embedding vector at a timestep.

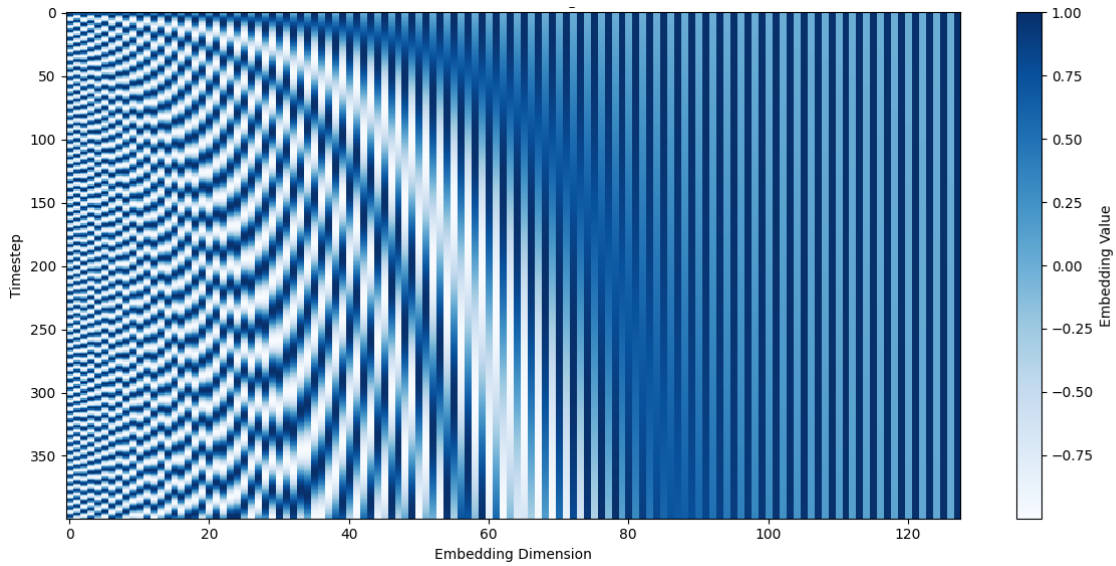


Figure 9: Sinusoidal embedding with 128 dimensions

3.2.3 Building blocks

3.2.3.1 Convolutional layers

Convolutional layers are the main building blocks of the neural networks used in this project. A convolution uses a kernel (or filter) which slides over the input data. At each position, the dot product is calculated between the kernel and the input region, producing a new value. These values result in a feature map highlighting the most important patterns in the data. The kernel weights are learnable, so the network learns which of the features are most relevant. Convolutions are in advantage of the fully connected layers because they need fewer parameters and are still able to learn meaningful representations. Convolutional layers also have important hyperparameters. The stride determines how far the kernel moves at each step. As an example, a stride of one shifts the kernel by one entry at a time, while a stride of four skips three entries between each step. Padding can also be added to keep the output

dimensions consistent. By adjusting the padding and the stride, the data can be downsampled. A convolution with a kernel size of 2x2 and stride of one can be seen in Figure 10.

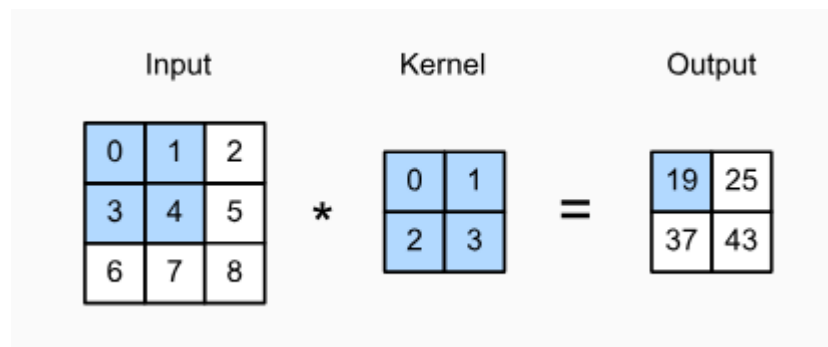


Figure 10: 2-dimensional convolution with a stride of one and no padding
(https://d2l.ai/chapter_convolutional-neural-networks/conv-layer.html)

To reverse the downsampling, transposed convolutions (also known as fractionally-strided convolutions) can be used. In a transposed convolution each input value is multiplied by the kernel, and the results are added to the output at the corresponding position. (Dumoulin and Visin, 2018)

Using a stride of one and a kernel size of 2x2 is visualized in Figure 11.

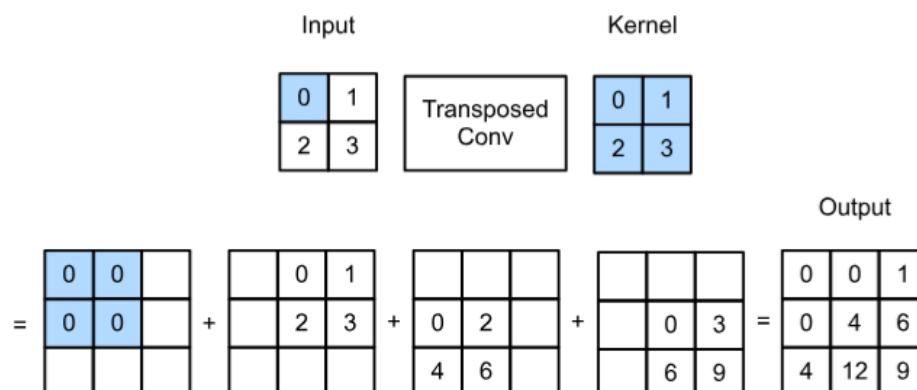


Figure 11: 2-dimensional transposed convolution with a stride of one and no padding
(https://d2l.ai/chapter_computer-vision/transposed-conv.html)

3.2.3.2 Receptive field

The receptive field of a model refers to the amount of input samples influence one single output unit. A larger receptive field means that the model can capture more long-term dependencies and broader patterns. In convolutional layers the receptive field can be increased using dilated convolutions, which work like normal convolutions, but the kernel is split up and contains gaps, increasing the effective kernel size to $(k - 1)d + 1$, where d is the dilation and k the kernel size (Dumoulin and Visin, 2018).

The receptive field of a model with convolutional layers at layer L can be calculated using the following formula:

$$RF = 1 + \sum_{i=1}^L (k_i - 1)d_i \prod_{j<i} s_j$$

Where d is the dilation k the kernel size and s the stride (Araujo, Norris and Sim, 2019).

3.2.3.3 Activation functions

Activation functions are a key building block for neural networks. They are typically added between every pair of layers. They introduce non-linearity to the neural network. This is crucial because without non-linearity, a neural net even with millions of parameters would reduce itself to a sequence of linear transformations. This would essentially be equivalent to linear regression, which would prevent the network from being able to learn complex data distributions. (“Activation function,” 2025)

The graphs of some common activation functions can be seen in Figure 12.

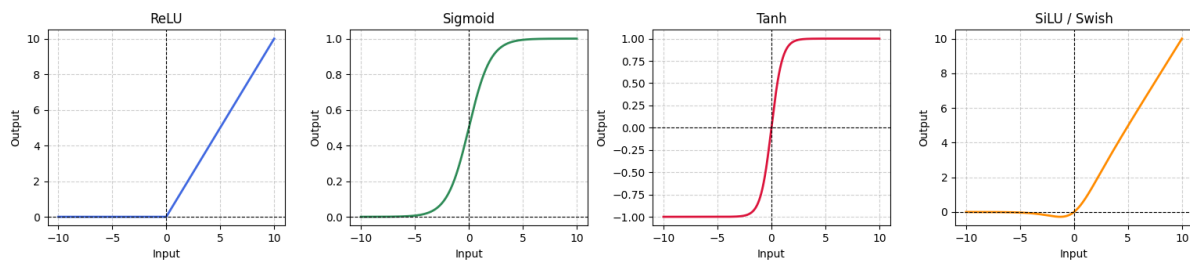


Figure 12: Common activation functions

3.2.4 Training neural networks

The main concepts enabling neural networks to learn are backpropagation⁵ and autograd⁶. Backpropagation calculates each parameter's error contribution in a neural network and adjusts them backward from output to input to improve performance. Autograd is a computational system that automatically runs backpropagation, it is an automatic differentiation engine. Both concepts are quite complex and handled internally by PyTorch, so no details will be mentioned here. Simply put, training neural networks works as follows: First the input is passed through the network to produce an output; this is the so-called forward

⁵ 3Blue1Brown, 2017: Backpropagation, intuitively | Deep Learning Chapter 3, [online]
<https://www.youtube.com/watch?v=IlG3gGewQ5U>

⁶ PyTorch: The Fundamentals of Autograd, [online]
https://docs.pytorch.org/tutorials/beginner/introyt/autogradyt_tutorial.html (accessed 22.9.2025).

pass. Then a loss is calculated between the predicted and true values. Finally, the backward pass with gradient descent and backpropagation is done to update the model's parameters. ("Backpropagation," 2025)

3.2.4.1 Gradient descent and optimization

To train a neural network, there is some kind of loss function (sometimes also referred to as cost function). This can for example be MSE⁷.

$$MSE = \frac{1}{N} \sum_{i=1}^N \left(x_{\text{pred}}^{(i)} - x_{\text{true}}^{(i)} \right)^2$$

The loss is calculated between the model input and model output. In linear regression the loss function is convex, making descent easy ("Linear regression," 2025). In neural networks on the other hand, the loss function often has many saddle points and local minima, which need to be overcome to reach a good solution - in practice a sufficient local minimum. Usually, it is not possible (or necessary) to reach the true global minimum. For this task gradient descent can be used, an optimizing algorithm. Starting at an arbitrary point, the loss function and its gradient at that point are calculated. The gradient indicates the direction and magnitude to adjust the model's parameters. The learning rate, or step size, a hyperparameter determining how far to move along the curve in one step, needs to be set for this to work. Setting it too large can lead to an unstable descent and the minimum could be overstepped, while setting it too small takes longer and could get the model stuck in a poor local minimum.

Gradient descent can formally be expressed as follows:

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

Where η is the learning rate, and ∇L the gradient of the loss function at the point θ_t . Using this, the next point θ_{t+1} can be calculated. (*What is Gradient Descent?* | IBM, 2021; "Gradient descent," 2025)

⁷ Wikipedia: Mean squared error, [online]
https://en.wikipedia.org/w/index.php?title=Mean_squared_error&oldid=1312394118.

The importance of the learning rate can be seen in the graph in Figure 13 where on the left the learning rate is too small and not able to overcome the saddle point and on the right the minimum is overstepped because the learning rate is too large.

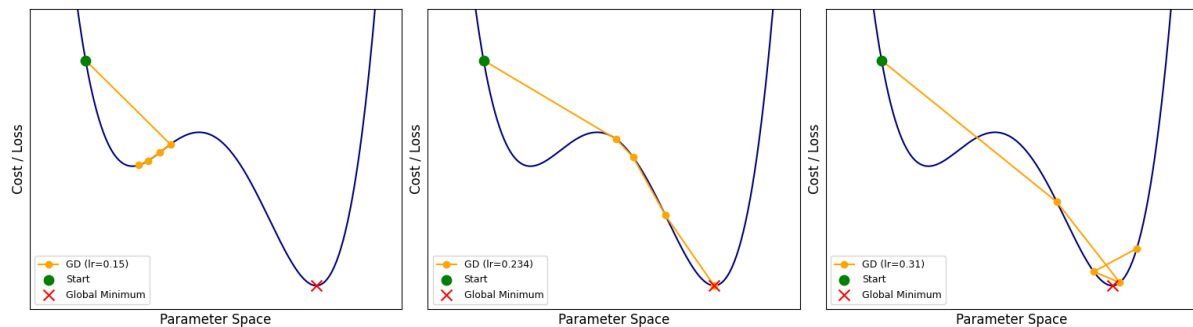


Figure 13: Gradient descent with different learning rates

As gradient descent can be unstable, especially in very high dimensional spaces, more advanced optimizers can be used. One of the most popular being Adam (Adaptive Moment Estimation). Adam works by setting the learning rate of each parameter individually and combining it with momentum. Adam introduces two parameters, β^1 and β^2 , which control how much past gradients and squared gradients influence the current update (momentum and variance estimation). The optimizer used in nearly all the following experiments was AdamW, which is an improved version of Adam. In Adam, the weight decay interacts with the adaptive learning rates, which can weaken its regularization effect. AdamW solves this by decoupling the weight decay from the learning rate. ("Stochastic gradient descent," 2025)

The optimization using AdamW can be seen in Figure 14.

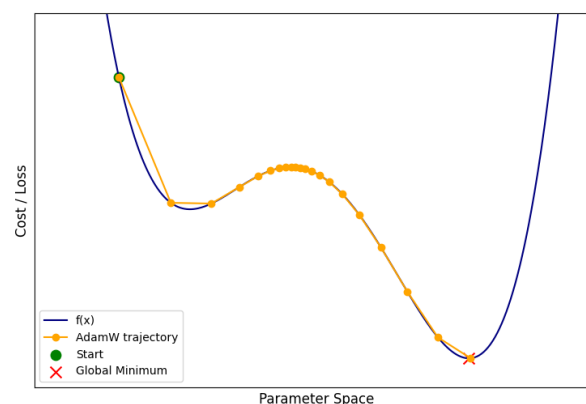


Figure 14: Gradient descent using AdamW

3.2.4.2 Hyperparameters

A neural network consists of thousands to millions of learnable parameters, giving it the ability to learn. Nevertheless, to train and build a model, the so-called hyperparameters need to be

set. They are not learned by the model. These are parameters describing the model and the learning process on a meta level. One of them is, for example, the previously mentioned learning rate, which determines the step size in the gradient descent. Others include the model layout in general (e.g. number of layers, kernel size of convolutions, and activation functions). Setting these up is critical. It requires many test runs to find the optimal values, as suboptimal values can lead to failure. Another difficulty is that the different parameters influence each other. This means that for a non-converging run the reasons might be insufficient model capacity, or simply an unsuitable learning rate may get the optimizer stuck in a bad local minimum. (“Hyperparameter (machine learning),” 2025)

There are also tools which can optimize hyperparameters, such as Optuna⁸. It works by running trials with different hyperparameter combinations, evaluating each run, and updating the search strategy. Optuna typically uses a kind of Bayesian optimization to propose the next set of parameters. However, even for such an optimizer, some controlling parameters must be set, such as the search range for each parameter. (Akiba *et al.*, 2019)

3.2.4.3 Other optimizations

In addition to the previously mentioned optimizations, there are a few more helpful techniques:

- Learning rate schedulers, which change the learning rate over time, often improve convergence. A common strategy is to gradually decrease the learning rate, which helps approach minima more precisely and prevents overshooting. One scheduler often used was CosineAnnealingWarmRestarts⁹, a learning rate scheduler, which periodically increases the learning rate again to escape non-optimal local minima. (Chugani, 2025)

A training run using CosineAnnealingWarmRestarts can be seen in Figure 15.

⁸ Optuna: Optimize Your Optimization, [online] <https://optuna.org/>.

⁹ PyTorch: CosineAnnealingWarmRestarts, [online]

https://docs.pytorch.org/docs/stable/generated/torch.optim.lr_scheduler.CosineAnnealingWarmRestarts.html (Accessed: September 22, 2025).

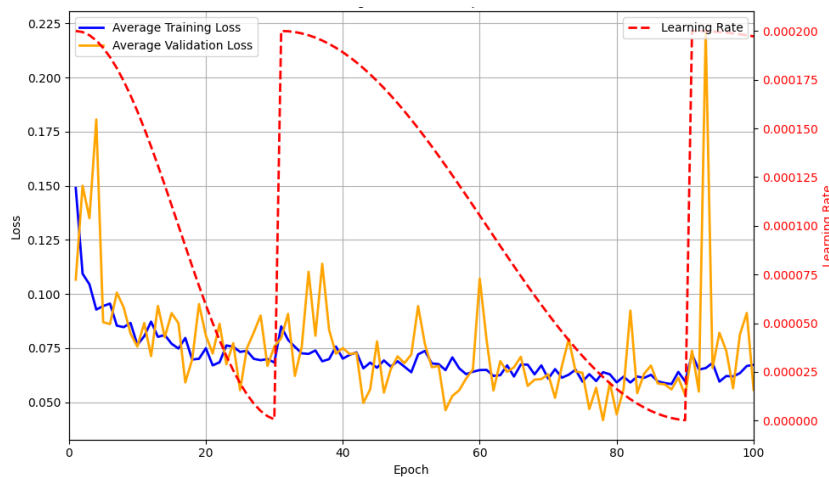


Figure 15: Training run using CosineAnnealingWarmRestarts

- Gradient clipping normalizes the gradients by preventing gradients from becoming too large, which could lead to unstable updates and eventually to numerical instability. (Zhang *et al.*, 2020)
- Batch size is a hyperparameter determining how many samples are processed at the same time. Larger batch sizes make training faster since only one backward pass is performed per batch. The loss is averaged across the batch dimension, smoothing the updates. The batch size is limited by GPU memory, which can be overcome by gradient accumulation¹⁰, which simulates a larger batch size. Larger batches also produce smoother gradient estimates, while smaller batches introduce more noise. (*Batch Size in Neural Network*, 13:39:05+00:00)

3.2.5 U-Net architecture

The neural network used in diffusion models is commonly a U-Net. The U-Net was initially proposed by Olaf Ronneberger in 2015 for biomedical image segmentation (Ronneberger, Fischer and Brox, 2015). It is a deep convolutional network built from convolutional and pooling layers¹¹. The architecture consists of an encoder path, a bottleneck, and a decoder path. From each layer in the encoder path, there is a skip connection to the corresponding layer in the decoder path. U-Nets are used because they process information at multiple

¹⁰ Hopsworx: Gradient Accumulation, [online] <https://www.hopsworx.ai/dictionary/gradient-accumulation> (Accessed: September 22, 2025).

¹¹ Wikipedia: Pooling layer, [online] https://en.wikipedia.org/w/index.php?title=Pooling_layer&oldid=1308048317.

scales, and the skip connections allow the network to preserve information from the noisy input, helping with the iterative denoising process.

When using spectrograms in diffusion, they can be regarded as images of shape $[channels, frequency\ bins, time]$ which are passed through a network using two-dimensional convolutions. Or, as proposed by Flavio Schneider, the frequency bins can be used as channels. In this case, the network receives an input of shape $[frequency\ bins, time]$ for which one-dimensional convolutions are used. The same approach applies to audio waves, where the input has shape $[audio\ channels, time]$. A one-dimensional network uses less memory and is more computationally efficient. (Ronneberger, Fischer and Brox, 2015; Schneider, 2023) The original network architecture by Ronneberger *et al.* is shown in Figure 16.

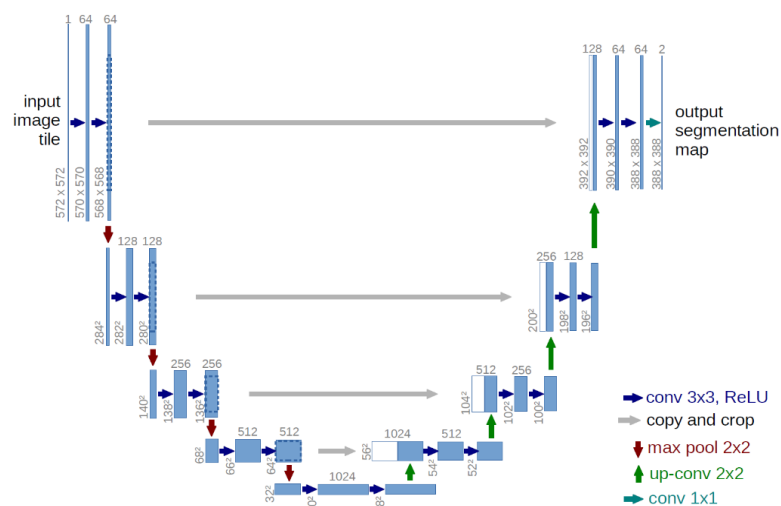


Figure 16: The original U-Net architecture (Ronneberger *et al.*, 2015)

4 Audio representation and processing

When working with audio, either raw waveforms can be used or the waveforms can be converted into spectrograms, which are a visual representation of frequency content over time. ("Spectrogram," 2025)

4.1 Waveform

By default, audio is represented as an amplitude signal evolving over time. For mono audio, this means the data is one-dimensional. At a sample rate¹² of 32 kHz, this means that there

¹² Sample rate: The number of samples taken from a continuous signal per second.

are 32000 samples representing one second of audio. Raw waveforms are challenging to learn from, because they are large in size and do not reflect perceptual aspects of hearing. However, working directly with raw waveforms has the benefit of not requiring any postprocessing. (Schneider, 2023)

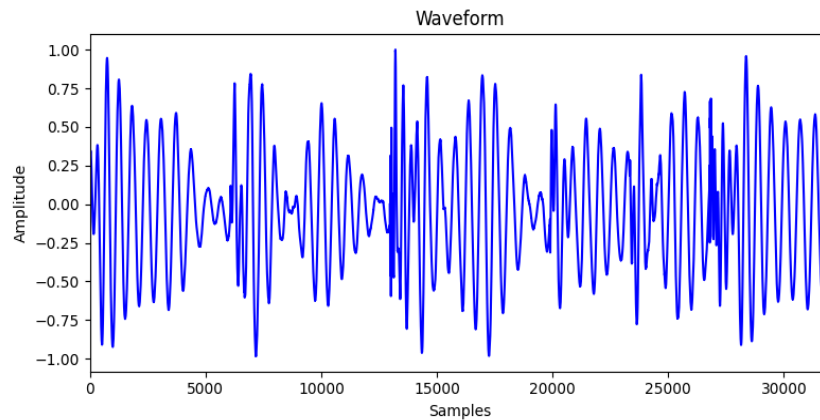


Figure 17: Amplitude-Time representation of audio

4.2 Spectrogram

Spectrograms are visual representations of audio. They show how the frequency content changes over time. They provide a more compact and interpretable form of audio data than raw waveforms, which is why they are often used for neural network training.

4.2.1 STFT spectrogram

Audio is commonly represented as a short-time Fourier transform (STFT) spectrogram, which is computed by applying the STFT algorithm¹³ to the waveform. STFT works by applying the fast Fourier transform (FFT) multiple times using a window function to isolate the frequencies of short intervals. STFT uses several important parameters that determine the output resolution. These include the FFT length, which defines the number of samples used for each transform, and the hop length, which specifies how far apart the analysis windows are placed in time. A smaller hop length increases the overlap between windows and results in smoother spectrograms. In practice, a Hann window was used to reduce spectral leakage, and zero-padding was applied when the FFT length exceeded the window length. A STFT spectrogram is a two-dimensional signal representation where each column corresponds to the frequency content of a short segment of audio. This results in a more compact representation compared

¹³ Wikipedia: Short-time Fourier transform, [online] https://en.wikipedia.org/w/index.php?title=Short-time_Fourier_transform&oldid=1305276667.

to raw waveforms. However, STFT produces two outputs: the phase and the magnitude of the signal. Looking at Figure 18, it can clearly be seen that the magnitude has a distinct structure, whereas the phase appears more random. Because of this, the phase is often disregarded, and the model is trained only on the magnitude. This is problematic, as the phase needs to be approximated either by using the Griffin-Lim algorithm¹⁴ or a second neural network, also known as vocoder.

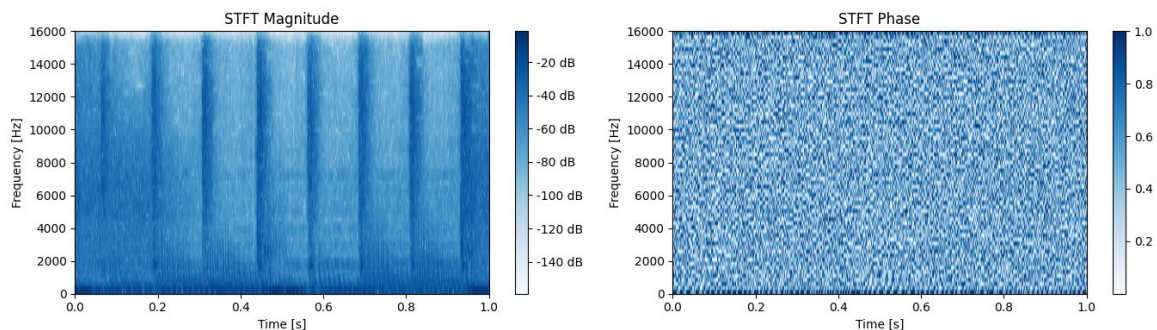


Figure 18: STFT magnitude and phase spectrograms (dB scale for visibility)

4.2.2 Mel spectrogram

The mel spectrogram also represents frequency content over time, like an STFT spectrogram. To compute a mel spectrogram, an STFT magnitude spectrogram is calculated, and mel-scaling is applied to it. Mel-scaling is based on human perception and divides the frequency axis into a fixed number of so-called frequency bins (usually 128 or fewer). The mel scale is designed to match human perception, by using more spacing frequency bins more densely at lower frequencies, where humans can perceive smaller pitch differences. This yields an even more compact representation with the dimension of *[frequency bins x time]*.

However, there is no direct inverse transformation from mel spectrogram to waveforms, which means that a vocoder must be used to reconstruct the audio waveform. (Schneider, 2023)

¹⁴ Griffin-Lim: A phase approximation algorithm.

The same four-second-long sample from Figure 18 can be seen as mel spectrogram in Figure 19.

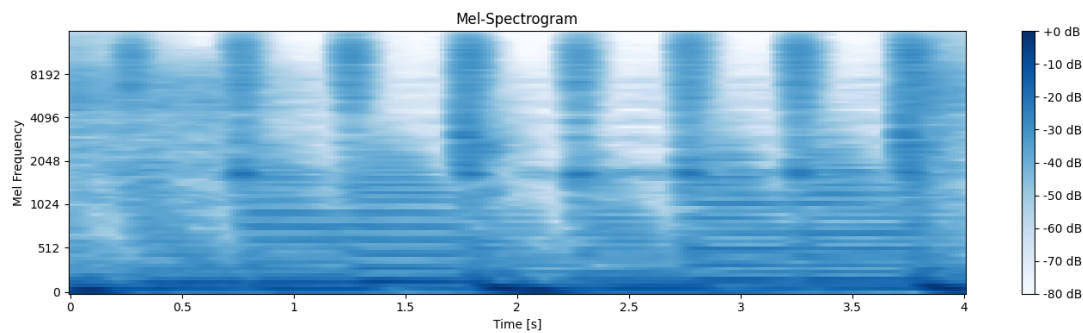


Figure 19: Mel-Spectrogram (dB scale for visibility)

4.2.3 Audio reconstruction

As previously mentioned, magnitude spectrograms and mel spectrograms are missing the phase information and cannot directly be converted back to waveforms. Reconstruction can either be done iteratively using Griffin-Lim, which estimates the missing phase, or with neural vocoders. Neural vocoders learn to directly synthesize waveforms from spectrograms. Vocoders also tend to produce better outputs, which is especially useful when there is no direct inverse transformation as it is the case with mel spectrograms. (Schneider, 2023)

5 Ethical considerations

As with all forms of generative AI, there are ethical considerations. The music generated by AI should not impersonate a particular artist or be overly influenced by a small number of artists. This is mitigated in this work by using DJ sets containing music from many different artists. In addition to this, the generated music will not be released on any streaming platforms. While the stylistic diversity is somewhat limited, as the model is trained on mostly techno and house tracks, ethical concerns related to genre bias are reduced. This aligns with general ethical guidelines for AI, which emphasize transparency, fairness, and responsibility when generating synthetic content (Jobin, Ienca and Vayena, 2019).

6 Experiment setup

6.1 Training data

The data used in the following experiments contains multiple DJ sets¹⁵ each around two hours. They contain techno and house tracks. This genre was chosen due to its clear rhythmic patterns, which help to evaluate the quality of the music generated, as well as personal preference. Some examples of the music can be found in the GitHub repository in the appendix. All the DJ sets were made by ADO¹⁶. Some of them are publicly available on SoundCloud.

6.1.1 Preprocessing

All models in this work were trained using a 32 kHz sample rate, providing a frequency range up to 16 kHz¹⁷, which is sufficient for good-quality audio while keeping memory usage lower than 44.1 kHz (CD-Quality) or even 48 kHz. The mentioned audio lengths are very short but were chosen due to computing limitations. (“Sampling (signal processing),” 2025)

Type	Sample Rate	Length	FFT Length	Hop Length	Mel Bins	Notes
STFT Spectrogram	32 kHz	2 ¹⁷ (~ 4s)	1023	256	-	-
Mel Spectrogram	32 kHz	2 ¹⁷ (~ 4s)	1023	256	96	-
Raw Waveforms	32 kHz	2 ¹⁷ (~ 4s)	-	-	-	Early waveform tests
Direct Wave Diffusion	32 kHz	2 ¹⁸ (~ 8s)	-	-	-	Random 4 s excerpts to increase diversity

Figure 20: Training data parameters

6.2 Test dataset

Besides the training dataset made from DJ sets, a small test set was also created. This dataset includes tracks not contained in the DJ sets and was used only for testing and evaluating model performance.

6.3 Hardware and software

Training neural networks requires a GPU (Graphics Processing Unit), which dramatically speeds up computation compared to a CPU. GPUs contain thousands of cores enabling them to perform many matrix and tensor multiplications simultaneously, which are essential in

¹⁵ DJ set: A selection of pre-recorded tracks mixed and more or less seamlessly transitioned by a DJ.

¹⁶ ADO on Soundcloud, [online] https://soundcloud.com/ado_sounds.

¹⁷ Wikipedia: Nyquist–Shannon sampling theorem, [online] https://en.wikipedia.org/w/index.php?title=Nyquist%E2%80%93Shannon_sampling_theorem&oldid=1309875996.

neural network training. The GPUs used, were rented from Paperspace¹⁸. Initially an NVIDIA A4000 GPU¹⁹ was used and later an NVIDIA A6000 GPU²⁰ or A100-80GB GPU²¹. In total, the project cost approximately 250 CHF. All of these GPUs are specifically designed for AI training and inference, as they have tensor and CUDA cores. Tensor cores are specialized for matrix multiplications, and CUDA cores handle parallel computation.

For the neural network framework, PyTorch²² was used, which integrates directly with CUDA²³, NVIDIA 's GPU compute framework. Audio processing was done using librosa²⁴.

7 Experiments

The goal of the following experiments is to generate short audio samples (4s) that match the training data in structure and content. Experiments were conducted using two approaches: spectrogram-based models (STFT or mel) and direct waveform models. This allows evaluation of model performance across different audio representations in terms of memory, training speed, and output quality. Model performance was assessed primarily through qualitative listening and visual inspection of spectrograms to identify whether there were recognizable structures.

7.1 Naive diffusion implementation

As a first step, I implemented the diffusion algorithms from scratch. The diffusion method used was DDPM, as it is the simplest to implement. The neural network was a simple 2-dimensional convolutional U-Net, I designed myself. I used 4-second audio clips converted to STFT magnitude spectrograms with a hop length of 288 and an FFT length of 480. In this implementation, there occurred many different issues such as a wrong activation function, a before the output, pushing the output in a wrong range. The data was also in an incorrect format, $[0, 1]$ instead of $[-1, 1]$, which is problematic because diffusion assumes input data centered around zero, and the noise must be in the same range as the input. Even after fixing

¹⁸ Paperspace, [online] <https://www.paperspace.com/>.

¹⁹ NVIDIA A4000, [online] <https://www.nvidia.com/en-us/products/workstations/rtx-a4000/>.

²⁰ Nvidia A6000, [online] <https://www.nvidia.com/en-us/products/workstations/rtx-a6000/>.

²¹ Nvidia A100-80GB, [online] <https://www.nvidia.com/en-us/data-center/a100/>.

²² PyTorch, [online] <https://pytorch.org/>.

²³ Wikipedia: CUDA, [online] <https://en.wikipedia.org/w/index.php?title=CUDA&oldid=1312021449>.

²⁴ Librosa, [online] <https://doi.org/10.5281/zenodo.15006942>.

these errors and many training runs with different hyperparameters, I did not get any meaningful outputs.

7.2 Diffusion based on other projects

As my diffusion didn't work, I looked up different diffusion projects, such as the explanation by Outlier²⁵. I adapted my implementation to his approach, which seemed to correct some errors in my first implementation. The loss decreased, and the model appeared to converge. This resulted in a first output, a spectrogram with some recognizable patterns, which can be seen in Figure 21.

I never converted it to audio, as I found it too noisy and expected better results in subsequent training runs, which did not occur.

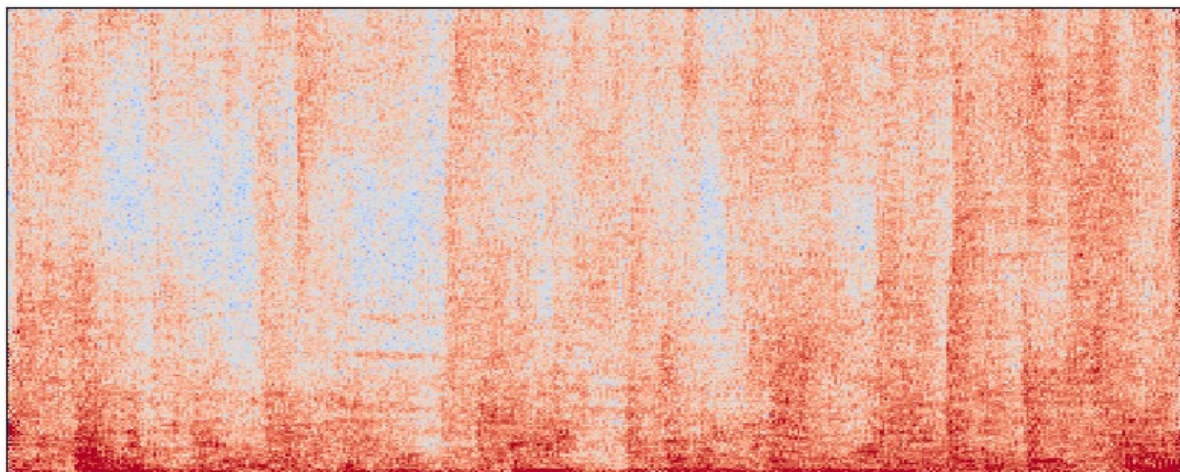


Figure 21: First diffusion output, frequency-time representation

I then tried many approaches, such as implementing the implicit sampling method and different architectures, but this didn't lead to any improvement. Therefore, I inspected my dataset, and I realized that I had previously normalized my audio by taking the minimum and maximum of all samples and then rescaling them to $[-1, 1]$. This was wrong, as I should have normalized each snippet individually to account for different perceived levels. Even though I fixed this error and ran many additional training iterations, no usable results were obtained. I went on to research more about neural network architecture, aided by a video from Luke Ditra²⁶, which revealed an additional fault in the network. The U-Net was missing the first skip

²⁵ Outlier, 2022: Diffusion Models | PyTorch Implementation, [online]
https://youtu.be/TBCRIWJtZU?si=1eLajS_ZRIgNWvc.

²⁶ Luke Ditra, 2024: Generative Image Diffusion from Scratch! PyTorch Deep Learning Tutorial, [online]
<https://www.youtube.com/watch?v=8SVV1FGFtik>.

connection going from the initial convolutional layer to the output layer. Fixing this improved training dynamics slightly but still did not produce usable outputs.

7.3 Audio diffusion PyTorch

After those failed attempts, I looked for similar projects and found the Archisound paper by Flavio Schneider from ETH. This paper also contains the audio diffusion PyTorch²⁷ library. Audio diffusion PyTorch is a collection of diffusion methods and networks based on Schneider's 2023 Master's thesis "Archisound: Audio generation with diffusion" (Schneider, 2023).

This paper seemed very promising, as the core idea aligned closely with my own work. The paper also describes the difficulties of audio diffusion and provides detailed explanations of the methods used. (Schneider, 2023)

7.3.1 Two-dimensional diffusion

I first tried training a two-dimensional diffusion model with input data of shape *[Batch, Channels, Frequency bins, Time]*. In the beginning, the model also seemed to be learning, but the outputs were not promising. The generated spectrograms were noisy, with no clear structures. Another problem with this method was the parameter count. The models quickly reached 100-200 million parameters, which meant long training times (up to 20 hours for 100 epochs). This drastically slowed down hyperparameter tuning and experimentation. Additionally, the large model size meant training with very small batch sizes, which further increased training time and destabilized training.

7.3.2 One-dimensional diffusion

To accelerate training and hopefully improve results, I then switched to one-dimensional networks. Here, I removed the extra channel dimension, so the input shape became *[Batch, Frequency bins, Time]*. This reduced the parameter count significantly, making training much faster. This change improved the results. The spectrograms now showed visible structures, as visible in Figure 22. However, when compared to real spectrograms, the outputs appeared blurry, also lacking long-term features. Long-term features would normally appear as horizontal lines over multiple time frames. When converting this to audio using Griffin-Lim,

²⁷ Audio-diffusion-pytorch, [online] <https://github.com/archinetai/audio-diffusion-pytorch>.

the reconstructed audio essentially was just random noise. The output of the model, converted using Griffin-Lim (GL), can be found here: <https://bit.ly/42E6WZM>.

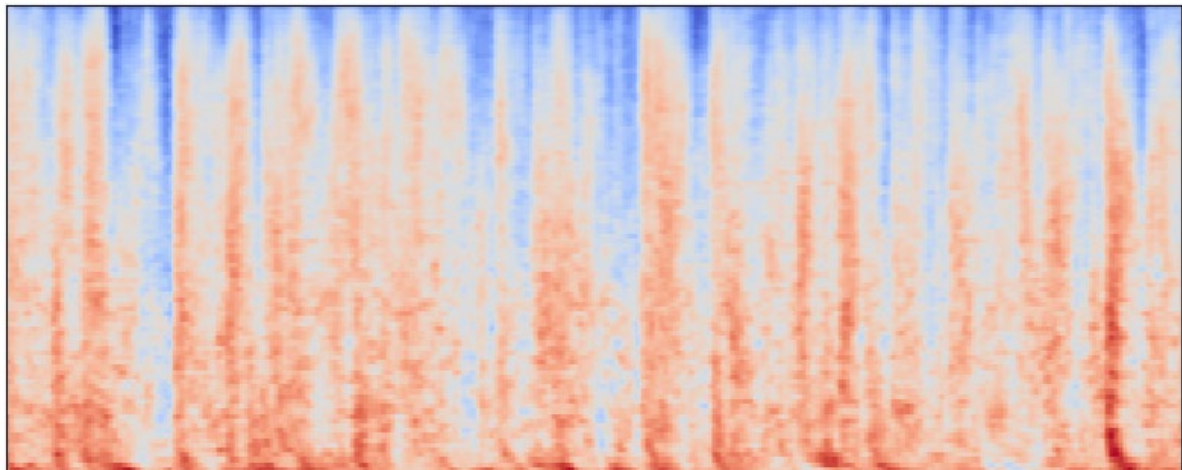


Figure 22: Diffusion output using audio diffusion PyTorch, frequency-time representation

I also received advice in machine learning communities doubting Griffin-Lim's ability to reconstruct anything. They suggested to use a vocoder instead.

At that time, I also met up with Driton Komani, Professor of Mathematics and Physics, and Christian Horvat, PhD (Artificial Intelligence), at FHNW to figure out why diffusion wasn't working properly. I received a lot of useful feedback, but unfortunately, it did not lead to significant improvements.

To simplify hyperparameter optimization, they suggested I use Optuna. This made training easier and more systematic, but it still did not improve results.

Finally, I also experimented with mel spectrograms of different sizes. However, besides speeding up training because of their smaller size, it did not noticeably affect the outputs.

7.4 Unconditional generation using the Moûsai architecture

Flavio Schneider also released a model together with Ojasv Kamal capable of generating music, called Moûsai. In that paper they described the exact architecture they used for unconditional music generation, which closely matched the goal of my project. I tried to implement that according to their setup, but the paper contained inconsistencies in some of the reported numbers. Which meant I could not train their exact model. This model worked directly with waveforms but additionally injected the spectrogram to improve output quality. I tried using Optuna to tune the parameters, but also with no usable results. (Schneider *et al.*, 2023)

A broad overview of the network architecture used for unconditional generation by Moûsai can be seen in Figure 23.

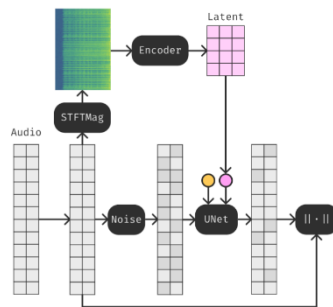


Figure 23: The Moûsai architecture, additionally encoding a STFT-magnitude spectrogram into a latent space and injecting it into the U-Net (Schneider et al., 2023)

7.5 Vocoders

Since I assumed that a vocoder could improve the output of my diffusion models, I implemented two different known approaches. The first one was a GAN based on MelGAN²⁸, a Generative Adversarial Network that converts mel spectrograms to audio. GANs consist of a generator and a discriminator, both neural networks. The generator produces a realistic output and the discriminator tries to find the fake data generated by the generator. This setup is very efficient, but complicates training, as both networks need to be balanced and training can easily collapse if one network gets significantly stronger than the other. I got this model to produce an acceptable output quality, when reconstructing audio from a ground-truth (non-AI generated) magnitude spectrogram. (“Generative adversarial network,” 2025)

I also trained a model based on HiFi-GAN²⁹, a more powerful and efficient GAN for vocoding than MelGAN. The main difference between the two is the network architecture. This model also produced better outputs, but when testing it together with the output of my diffusion model results weren’t better than the ones using Griffin-Lim. This led me to conclude that my diffusion outputs were statistically too different from the original spectrograms, so the vocoders could not reconstruct them sufficiently.

²⁸ Kumar, K. et al., 2019: MelGAN: Generative Adversarial Networks for Conditional Waveform Synthesis, [online] <https://doi.org/10.48550/arXiv.1910.06711>.

²⁹ Kong, J et al., 2020: HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis, [online] <https://doi.org/10.48550/arXiv.2010.05646>.

The output of the audio diffusion PyTorch model, converted using HiFi-GAN, can be found here: <https://bit.ly/42E6WZM>.

7.6 VQ-VAE and transformer

At that point I went on to try some completely different approaches, one of which was based on Jukebox³⁰. Jukebox uses a vector quantized variational autoencoder (VQ-VAE) to generate a tokenized version of the audio with a limited number of tokens, e.g. 1024 or 2048. The VQ-VAE learns how to represent a waveform using the learned tokens and to reconstruct the original waveform from them. This is very efficient in theory, as it drastically reduces the dimensionality of the audio, but it is challenging to ensure the tokens capture meaningful audio features. After generating those tokens, a transformer (GPT-style³¹) is used to autoregressively predict the next token, and by giving the model a starting token it should be able to generate new samples. Although the VQ-VAE reconstructed audio well, the token representations were not meaningful, so training the transformer did not yield useful outputs. The architecture of the VQ-VAE combined with a transformer can be seen in Figure 24. (Oord, Vinyals and Kavukcuoglu, 2018)

³⁰ Dhariwal, P. et al., 2020: Jukebox: A Generative Model for Music, [online] <https://doi.org/10.48550/arXiv.2005.00341>.

³¹ Wikipedia: Generative pre-trained transformer, [online] https://en.wikipedia.org/w/index.php?title=Generative_pre-trained_transformer&oldid=1309626917.

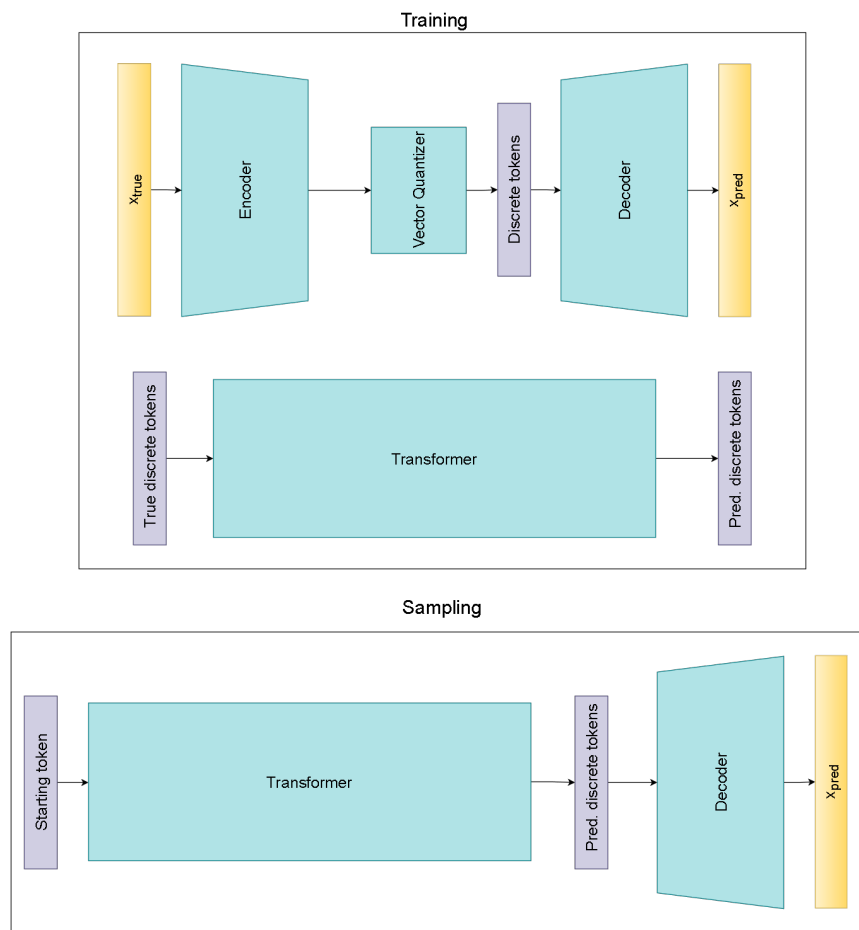


Figure 24: VQ-VAE + Transformer architecture overview

7.7 VAE

As the reconstruction of the VQ-VAE was quite good, I also tried to train a normal variational autoencoder (VAE). It takes in the audio waveform and encodes it into a latent probability distribution (mean and variance tensor). During training, the loss combines the reconstruction loss (e.g. MSE) and the Kullback-Leibler (KL) divergence³², encouraging the latent distribution to approximate a standard normal distribution $\mathcal{N}(0, I)$. This helps when sampling as new latent vectors can be drawn from the standard normal distribution to generate new audio. The architecture of a VAE can be seen in Figure 25. However, these models also have their issues; if the KL and the reconstruction loss are not weighted properly, the latent space can collapse. Even though the model achieved good reconstruction quality, I was not able to generate new data because I was not able to get the latent space to converge to a meaningful distribution. (Kingma and Welling, 2019)

³² Wikipedia: Kullback-Leibler divergence,[online]
https://en.wikipedia.org/w/index.php?title=Kullback%E2%80%93Leibler_divergence&oldid=1309977014.

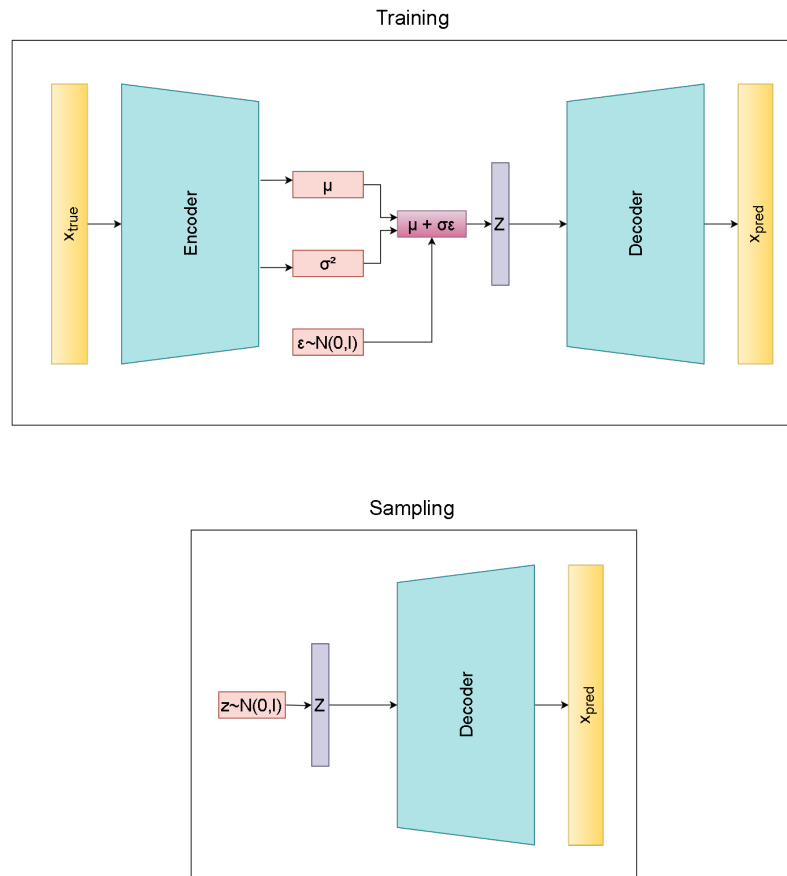


Figure 25: VAE architecture overview

7.8 Direct wave diffusion

After all these failed attempts, I decided to start over completely and try once more to use diffusion to generate new music. I call this approach direct wave diffusion, as the diffusion process is applied directly to raw audio waveforms rather than intermediate representations such as spectrograms. Instead of using audio-diffusion-pytorch or any other libraries, I implemented everything again from scratch using PyTorch only. I rebuilt the diffusion class and thoroughly tested it to ensure the correctness of the math.

I chose v-objective diffusion again, as it seemed more stable than DDPM or DDIM in the previous experiments, converging more easily and providing better results. During testing, I found a few major and minor issues which might have caused previous failures, such as wrong indexing and a reversed sampling trajectory. In my v-objective implementation, I previously went from no noise to full noise, whereas diffusion requires starting from full noise to a clean signal. I also decided to work with audio waveforms directly, eliminating the need to convert spectrograms back to audio, requiring additional models for converting spectrograms back to audio and introducing artifacts.

7.8.1 Network architecture and activations

The other relevant part is the neural network: a U-Net. I based it on the original U-Net implementation but replaced the convolutions at each level of the U-Net with residual blocks (ResBlock). I also built the bottleneck using ResBlocks. ResBlocks contain a skip connection, adding the input back to the output of the block. This improves gradient flow and preserves information. (He *et al.*, 2015)

For upsampling, I used a transposed convolution layer and a normal convolution layer. I added time conditioning in each residual block to provide the model with information about the current timestep. In practice, this is done by embedding the timestep using sinusoidal embeddings.

Additionally, the embedded timestep is passed through a multilayer perceptron (MLP)³³, with an additional linear layer at each injection point to project the timestep into the right dimension.

As an activation function, I used Sigmoid Linear Unit (SiLU), which is defined as follows:

$$SiLU = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

The graph of SiLU is shown in Figure 12.

SiLU is beneficial over ReLU because it provides a smooth transition into the negative region, preventing neurons from becoming permanently inactive (“dead neurons”). The SiLU function outputs small but nonzero outputs for negative inputs. This improves gradient flow during backpropagation, which helps stabilize training and can lead to better convergence in larger networks, such as mine.

7.8.2 Receptive field and convolutions

Another important factor, which I had not considered before, was the receptive field. This is the amount of input data points that are used to create a single output unit. Previously, I always used convolutions with kernel size three or four and no dilation, as these were the parameters used in all the tutorials etc. This worked fine for them as their inputs usually had a maximum of 512 data points in the time domain. My inputs, on the other hand, had a size of 131072 (2^{17}) which is much larger than that, so the receptive field was too small to capture

³³ Wikipedia: Multilayer perceptron, [online]
https://en.wikipedia.org/w/index.php?title=Multilayer_perceptron&oldid=1305076934.

any patterns. Realizing that, I knew that I had to drastically increase the receptive field in my model. In direct wave diffusion version 6 (v6), my final architecture, the receptive field has a size of ~ 47500 , meaning at a sample rate of 32 kHz this corresponds to 1.45 s.

Using that information, I built my network and found the sweet spot of efficiency and receptive field with a kernel size of eleven for all convolutions and an exponentially increasing dilation size for the convolutions in the ResBlocks. And in the upsampling path the dilation sizes decrease exponentially. So, in my network with a depth of 4 layers, the dilation sizes are [1, 2, 4, 8] in the encoder then [16, 32] in the bottleneck and [8, 4, 2, 1] in the decoder. I did a few tests before coming up with these parameters. I achieved some results already with version three of the wave diffusion, which was a very small network with around 2 million parameters. It used a kernel size of 9 and produced bad outputs, but proved that the algorithms were working properly.

This can be heard at: https://bit.ly/DirectWaveDiffusion_v3_sample.

7.8.3 Results and analysis

Therefore, I increased the network capacity and changed some other parameters. I also fixed some minor bugs in version four and five and then came up with version six. The architecture can be seen in Figure 26 and Figure 27 (The legend for Figure 26 can be found in Figure 27). The tensor sizes are indicated after every layer.

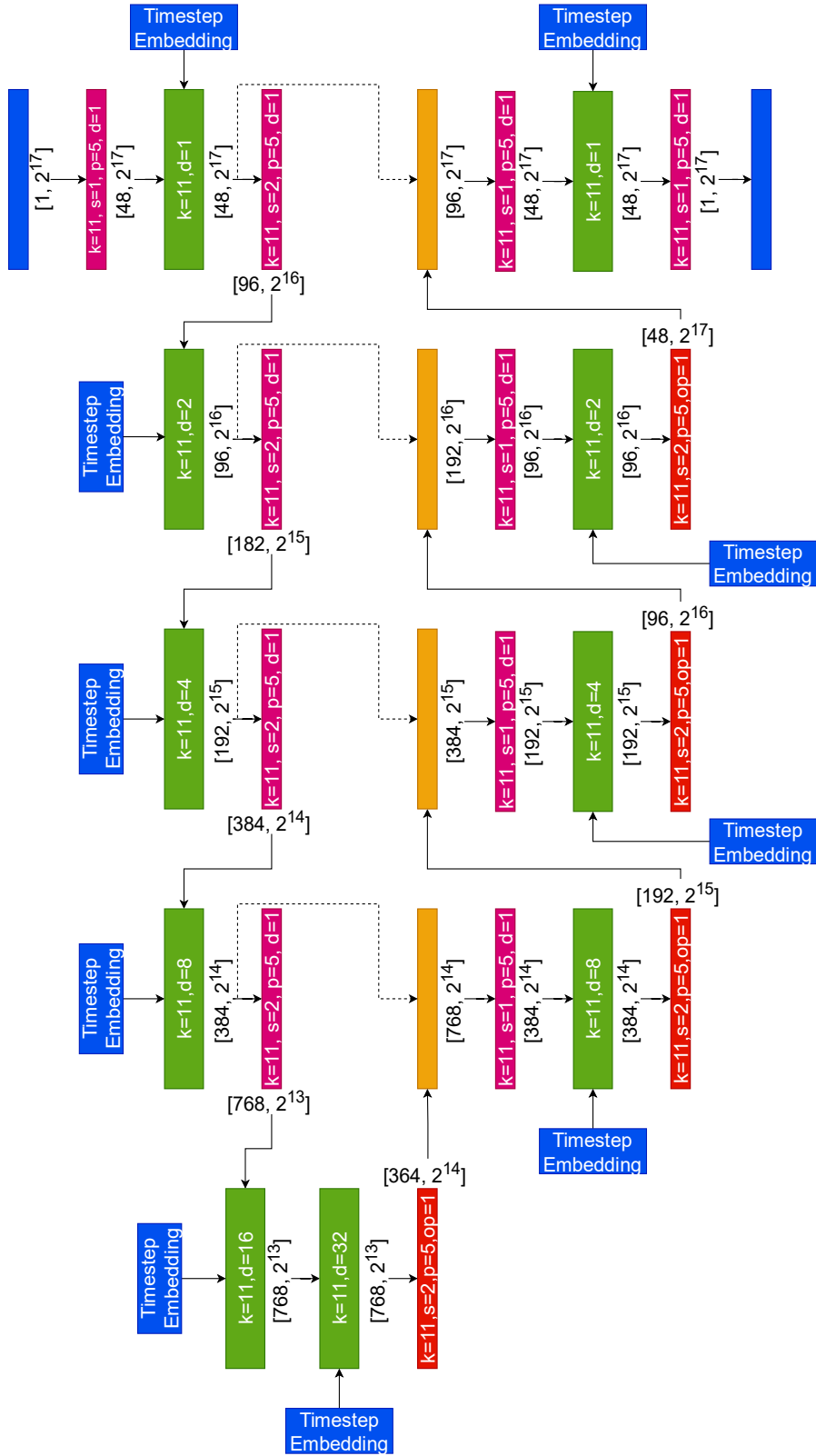


Figure 26: The Architecture of the exact U-Net used in my direct wave diffusion implementation. The batch dimension is disregarded for simplicity

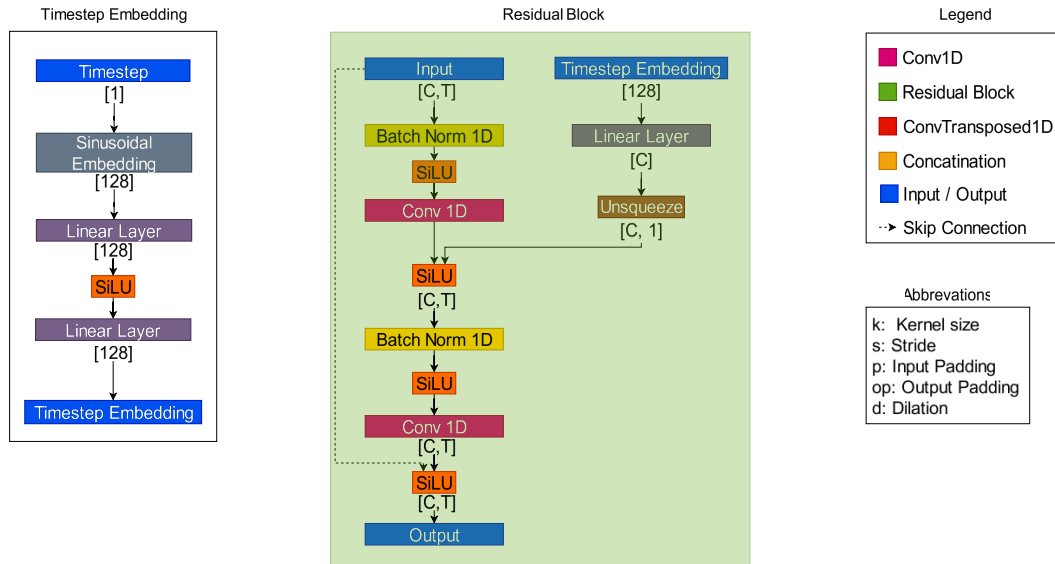


Figure 28: Additional architecture notes for wave diffusion v6

The Training parameters for this network can be seen in Figure 28. After some additional tuning I increased the learning rate to 6.1×10^{-4} .

Parameter	Value
Batch size	16 or 32 (depending on GPU)
Learning rate	2×10^{-4}
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.99$)
Scheduler	CosineAnnealingWarmRestarts ($T_0 = 30$, $\eta_{min} = 1 \times 10^{-7}$, $T_{mult} = 2$)
Diffusion method	V-objective
Loss	MSE + Multiband Filter MAE (weights: MSE = 1, filter bands = [0.1, 0.15, 0.2, 0.1])
Filterbank	Bands [0, 300, 3kHz, 8kHz, 16kHz], 512 taps

Figure 27: Training parameters for direct wave diffusion v6

As can be heard in the samples of direct wave diffusion v6: <https://bit.ly/46nVVhz>, the model only seemed to reproduce the percussive parts of the audio. This is because those parts are often louder than the melodic parts and have more energy. This means that they play a more important role in the loss function of the model. In this case, this even means that the model converges but can completely ignore the melodic parts to do so.

To overcome this issue, I thought of splitting the audio into multiple frequency bands using a pseudo quadrature mirror filter bank (PQMF)³⁴ and training the model on individual bands. Using the bands as individual channels should, in theory, give more importance to the higher

³⁴ Smith, J. O. 2011: Pseudo-QMF Cosine Modulation Filter Bank, [online book] https://ccrma.stanford.edu/~jos/sasp/Pseudo_QMF_Cosine_Modulation_Filter.html (Accessed: September 22, 2025).

bands. This approach did not yield any results, I'm uncertain why, but the model architecture may have been unsuitable for multi-band training. I also tried a static filter bank³⁵ with individual band-pass filters³⁶ instead of the PQMF, but this did not work either. Therefore, I went back to using just one band. To get some benefit from multiple bands still, I modified my model's loss function. Previously I only used MSE between the true and the predicted velocity. Now I added a second term to the loss. I reconstructed the original waveform from the true and predicted velocity. I then applied my filter bank to it, splitting the waveforms into four bands with the following frequency ranges with a filter order³⁷ of 512:

Band 1: 0 Hz to 300 Hz

Band 2: 300 Hz to 3 kHz

Band 3: 3 kHz to 8 kHz

Band 4: 8 kHz to 16 kHz

The frequency responses³⁸ of the individual band-pass filters are shown in Figure 29. And an applied version on a sample can be heard at: <https://bit.ly/AppliedFilterbank>.

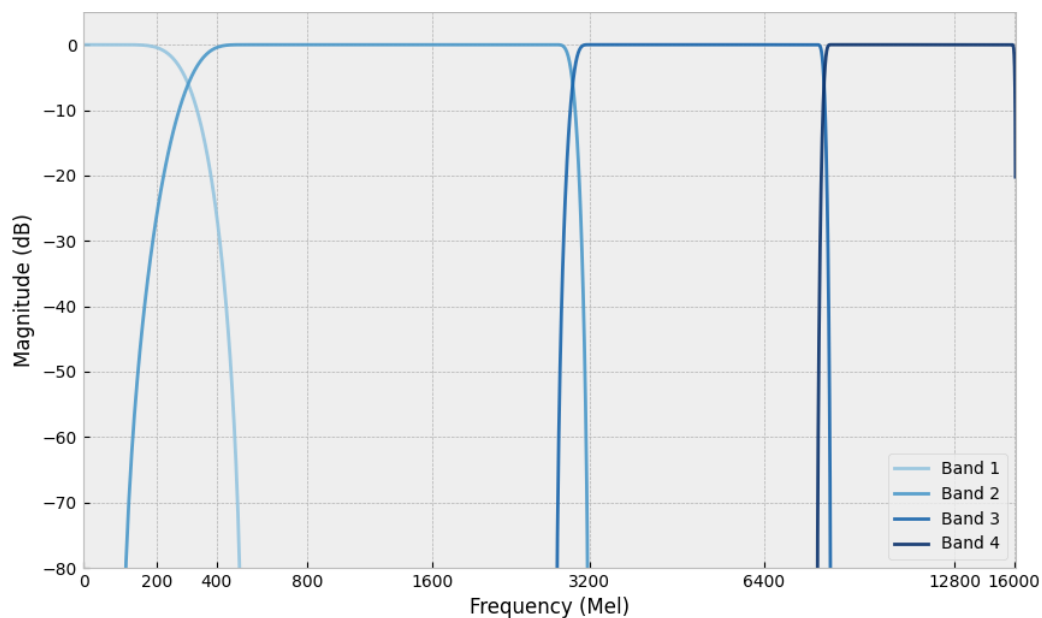


Figure 29: Frequency response of the filter bank

³⁵ Wikipedia: Filter bank, [online] https://en.wikipedia.org/w/index.php?title=Filter_bank&oldid=1309454093.

³⁶ Wikipedia: Band-pass filter, [online] https://en.wikipedia.org/w/index.php?title=Band-pass_filter&oldid=1293780847.

³⁷ Filter order (taps): number of coefficients in the filter; higher values result in a steeper cutoff and more precise frequency separation.

³⁸ Wikipedia: Frequency response, [online] https://en.wikipedia.org/w/index.php?title=Frequency_response&oldid=1312270906.

I then calculate the mean absolute error (MAE)³⁹ for each band.

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_{\text{pred}}^{(i)} - x_{\text{true}}^{(i)}|$$

The MAE of each band then is weighted to give more importance to the higher frequency bands. The weights used from band 1-4 were [0.1, 0.15, 0.2, 0.1]. This ensures that the MSE of the velocities still dominates the loss, as it is more relevant. Also, I gave a lower weight to the fourth band, as these frequencies usually don't contain that much relevant information. This loss modification helped with my issue as the model started producing some higher frequency content even though mostly some random notes and not any recognizable melodies. Another factor for sampling quality is the number of diffusion steps. The more steps, the clearer the audio; but if too many steps are used (500-1000), the model starts to remove melodic content from the generated samples. More steps also increases the sampling time. Samples generated with different numbers of diffusion steps can be found here: <https://bit.ly/46nVVhz>. Furthermore, there are also some seeded samples available at that link. These were generated by taking a sample from the test set, applying forward diffusion for a fixed number of steps, and then reversing the process through backward diffusion. In the provided examples, 90 forward and backward steps were used. The diffusion v6 model was trained for ~70000 iterations⁴⁰, taking around 30-35 hours.

In addition, the GitHub repository contains an alternative model that first generates audio at 16 kHz and then upsamples it to 32 kHz using a second model. While the upsampling model achieved very high quality on its own, combining it with diffusion did not noticeably improve the final audio quality.

7.8.4 Limitations and possible improvements

The outputs of my model are still not rhythmically coherent. At the time, the model had no way of determining whether the generated samples are rhythmically correct. Addressing this issue would be complex, as it would require additional information about the input data, such as the BPM⁴¹ or the position of the sample within a musical bar. Short samples make this particularly challenging, since they often do not contain enough information for an algorithm

³⁹ Wikipedia: Mean absolute error, [online]

https://en.wikipedia.org/w/index.php?title=Mean_absolute_error&oldid=1276071917.

⁴⁰ One iteration includes one backward pass.

⁴¹ BPM (Beats Per Minute): a measure of tempo, indicating how many beats occur in one minute.

to reliably estimate the BPM. One potential solution would be to calculate the BPM from longer audio segments before cutting them into short samples. However, the problem would remain during training, as the model would still need to calculate the BPM on short samples. Another possible approach would be to align the training samples to beats or bars rather than at an arbitrary time position. While there are tools available for beat or bar alignment, they typically require a considerable amount of manual labor. This contradicts my prerequisite that the models should not depend on manual preprocessing.

Incorporating temporal conditioning, such as embeddings for BPM or beat position, could be an interesting approach for future work.

8 Reflection

The work on this matura project taught me a lot about machine learning, especially generative deep learning. There were many complications, such as incorrectly normalized data or mistakes in the v-objective formulation of diffusion, and for a long time no usable results were obtained. This project showed me how important iterative experimentation is, and how crucial tuning architectures and hyperparameters are for achieving meaningful outcomes. Another important aspect I realized is how time-consuming the entire process is. Training deep learning models required many hours due to the complexity of the models, and the limited batch sizes that could be used on the available hardware, prolonging experimentation. It also gave me insight into the complexity of AI research, the huge amount of time and resources needed for training deep learning models, and how strongly results depend on having a good dataset with proper preprocessing. Furthermore, I learned that many machine learning papers omit certain implementation or hyperparameter, making reproduction of their experiments difficult. I'm satisfied that I managed to produce some results in the end, even though they were noisy. If I were to do the project again, I would start with simpler experiments, such as diffusion on a small standard dataset, to better understand the functionality of the models before directly attempting full music generation.

If I were to continue this work, I would focus on a few different points:

- spend more time researching or building a compression model, such as a VAE, as that would increase training speed and make experimentation much faster

- investigate a better preprocessing pipeline, possibly extracting extra features that could be embedded in the neural network to improve output quality
- explore conditional generation, for example conditioning the model on the previous sample to enable sequential generation and therefore longer outputs.

The leading question of this project was: *“How can I use denoising diffusion probabilistic models in deep learning to create music based on DJ sets as training data?”*

Through this work, I found that diffusion models can indeed be used to generate new audio resembling the training material when trained directly on raw waveforms. While the results were not yet rhythmically coherent, they demonstrated that diffusion can model musical textures without requiring manual annotation or preprocessing. This shows that the approach is feasible and lays the groundwork for more advanced models that could generate structured and musically consistent pieces.

9 References

9.1 Bibliography

- “Activation function” (2025) *Wikipedia*.
https://en.wikipedia.org/w/index.php?title=Activation_function&oldid=1310047582
(Accessed: September 8, 2025).
- Akiba, T. *et al.* (2019) “Optuna: A Next-generation Hyperparameter Optimization Framework.” *arXiv*. <https://doi.org/10.48550/arXiv.1907.10902> (Accessed: September 8, 2025).
- Araujo, A., Norris, W. and Sim, J. (2019) “Computing Receptive Fields of Convolutional Neural Networks,” *Distill*, 4(11), p. e21. <https://doi.org/10.23915/distill.00021>.
- “Backpropagation” (2025) *Wikipedia*.
<https://en.wikipedia.org/w/index.php?title=Backpropagation&oldid=1309329636>
(Accessed: September 24, 2025).
- Batch Size in Neural Network* (13:39:05+00:00) *GeeksforGeeks*.
<https://www.geeksforgeeks.org/deep-learning/batch-size-in-neural-network/>
(Accessed: September 8, 2025).
- Chugani, V. (2025) “A Gentle Introduction to Learning Rate Schedulers,”
MachineLearningMastery.com, 16 May. <https://machinelearningmastery.com/a-gentle-introduction-to-learning-rate-schedulers/> (Accessed: September 8, 2025).
- Dumoulin, V. and Visin, F. (2018) “A guide to convolution arithmetic for deep learning.” *arXiv*.
<https://doi.org/10.48550/arXiv.1603.07285> (Accessed: September 8, 2025).
- “Generative adversarial network” (2025) *Wikipedia*.
https://en.wikipedia.org/w/index.php?title=Generative_adversarial_network&oldid=1312281575 (Accessed: September 20, 2025).
- “Gradient descent” (2025) *Wikipedia*.
https://en.wikipedia.org/w/index.php?title=Gradient_descent&oldid=1307034450
(Accessed: September 8, 2025).
- He, K. *et al.* (2015) “Deep Residual Learning for Image Recognition.” *arXiv*.
<https://doi.org/10.48550/arXiv.1512.03385> (Accessed: September 21, 2025).
- Ho, J., Jain, A. and Abbeel, P. (2020) “Denoising Diffusion Probabilistic Models.” *arXiv*.
<https://doi.org/10.48550/arXiv.2006.11239> (Accessed: June 6, 2025).
- Huang, R. *et al.* (2022) “FastDiff: A Fast Conditional Diffusion Model for High-Quality Speech Synthesis.” *arXiv*. <https://doi.org/10.48550/arXiv.2204.09934> (Accessed: September 21, 2025).
- “Hyperparameter (machine learning)” (2025) *Wikipedia*.
[https://en.wikipedia.org/w/index.php?title=Hyperparameter_\(machine_learning\)&oldid=1299447463](https://en.wikipedia.org/w/index.php?title=Hyperparameter_(machine_learning)&oldid=1299447463) (Accessed: September 8, 2025).
- Jobin, A., Ienca, M. and Vayena, E. (2019) “Artificial Intelligence: the global landscape of ethics guidelines,” 2019 [Preprint].
- Kingma, D.P. and Welling, M. (2019) “An Introduction to Variational Autoencoders,”
Foundations and Trends® in Machine Learning, 12(4), pp. 307–392.
<https://doi.org/10.1561/22000000056>.

- Kong, J., Kim, J. and Bae, J. (2020) "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis." arXiv. <https://doi.org/10.48550/arXiv.2010.05646> (Accessed: September 20, 2025).
- "Linear regression" (2025) *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Linear_regression&oldid=1309707073 (Accessed: September 8, 2025).
- "Neural network (machine learning)" (2025) *Wikipedia*. [https://en.wikipedia.org/w/index.php?title=Neural_network_\(machine_learning\)&oldid=1311533855](https://en.wikipedia.org/w/index.php?title=Neural_network_(machine_learning)&oldid=1311533855) (Accessed: September 16, 2025).
- Nichol, A. and Dhariwal, P. (2021) "Improved Denoising Diffusion Probabilistic Models." arXiv. <https://doi.org/10.48550/arXiv.2102.09672> (Accessed: June 6, 2025).
- Oord, A. van den, Vinyals, O. and Kavukcuoglu, K. (2018) "Neural Discrete Representation Learning." arXiv. <https://doi.org/10.48550/arXiv.1711.00937> (Accessed: September 20, 2025).
- Ronneberger, O., Fischer, P. and Brox, T. (2015) "U-Net: Convolutional Networks for Biomedical Image Segmentation." arXiv. <https://doi.org/10.48550/arXiv.1505.04597> (Accessed: June 7, 2025).
- "Sampling (signal processing)" (2025) *Wikipedia*. [https://en.wikipedia.org/w/index.php?title=Sampling_\(signal_processing\)&oldid=1310449049](https://en.wikipedia.org/w/index.php?title=Sampling_(signal_processing)&oldid=1310449049) (Accessed: September 19, 2025).
- Schneider, F. (2023) "Archisound: Audio generation with diffusion," 2023 [Preprint]. <https://arxiv.org/pdf/2301.13267>.
- Schneider, F. *et al.* (2023) "Moûsai: Text-to-Music Generation with Long-Context Latent Diffusion." arXiv. <https://doi.org/10.48550/arXiv.2301.11757> (Accessed: June 6, 2025).
- Sohl-Dickstein, J. *et al.* (2015) "Deep Unsupervised Learning using Nonequilibrium Thermodynamics." arXiv. <https://doi.org/10.48550/arXiv.1503.03585> (Accessed: June 6, 2025).
- Song, J., Meng, C. and Ermon, S. (2020) "Denoising Diffusion Implicit Models." arXiv. <https://doi.org/10.48550/arXiv.2010.02502> (Accessed: June 6, 2025).
- "Spectrogram" (2025) *Wikipedia*. <https://en.wikipedia.org/w/index.php?title=Spectrogram&oldid=1291897572> (Accessed: June 6, 2025).
- "Stochastic gradient descent" (2025) *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Stochastic_gradient_descent&oldid=1309164477#Adam (Accessed: September 8, 2025).
- "Unsupervised learning" (2025) *Wikipedia*. https://en.wikipedia.org/w/index.php?title=Unsupervised_learning&oldid=1308118963 (Accessed: September 24, 2025).
- Vaswani, A. *et al.* (2023) "Attention Is All You Need." arXiv. <https://doi.org/10.48550/arXiv.1706.03762> (Accessed: September 21, 2025).
- What is Gradient Descent? | IBM* (2021). <https://www.ibm.com/think/topics/gradient-descent> (Accessed: September 8, 2025).
- Zhang, J. *et al.* (2020) "Why gradient clipping accelerates training: A theoretical justification for adaptivity." arXiv. <https://doi.org/10.48550/arXiv.1905.11881> (Accessed: September 24, 2025).

9.2 Table of figures

Figure 1: Forward and backward diffusion process	6
Figure 2: Linear vs. cosine noise schedule, visualized with images for clarity	7
Figure 3: DDPM training algorithm (adapted from Ho et al., 2020)	8
Figure 4: DDPM sampling algorithm (adapted from Ho et al., 2020)	8
Figure 5: DDIM sampling algorithm	10
Figure 6: V-Objective diffusion circle (Schneider, 2023)	10
Figure 7: V-Objective training algorithm	11
Figure 8: V-Objective sampling algorithm	12
Figure 9: Sinusoidal embedding with 128 dimensions	13
Figure 10: 2-dimensional convolution with a stride of one and no padding (https://d2l.ai/chapter_convolutional-neural-networks/conv-layer.html)	14
Figure 11: 2-dimensional transposed convolution with a stride of one and no padding (https://d2l.ai/chapter_computer-vision/transposed-conv.html)	14
Figure 12: Common activation functions	15
Figure 13: Gradient descent with different learning rates	17
Figure 14: Gradient descent using AdamW	17
Figure 15: Training run using CosineAnnealingWarmRestarts	19
Figure 16: The original U-Net architecture (Ronneberger et al., 2015)	20
Figure 17: Amplitude-Time representation of audio	21
Figure 18: STFT magnitude and phase spectrograms (dB scale for visibility)	22
Figure 19: Mel-Spectrogram (dB scale for visibility)	23
Figure 20: Training data parameters	24
Figure 21: First diffusion output, frequency-time representation	26
Figure 22: Diffusion output using audio diffusion PyTorch, frequency-time representation ..	28
Figure 23: The Moûsai architecture, additionally encoding a STFT-magnitude spectrogram into a latent space and injecting it into the U-Net (Schneider et al., 2023)	29
Figure 24: VQ-VAE + Transformer architecture overview	31
Figure 25: VAE architecture overview	32
Figure 26: The Architecture of the exact U-Net used in my direct wave diffusion implementation. The batch dimension is disregarded for simplicity	35

Figure 28: Training parameters for direct wave diffusion v6	36
Figure 27: Additional architecture notes for wave diffusion v6.....	36
Figure 29: Frequency response of the filter bank.....	37

10 Acknowledgements

I would like to thank Mrs. Steiner-Curtis for her guidance and supervision throughout this project. I am also grateful to Adrian Meyer for providing the DJ sets and assisting with proofreading, as well as to Matthias Meyer for additional proofreading. Finally, I would like to thank Driton Komani and Christian Horvat at FHNW for their helpful advice and insights.

11 Appendix

11.1 GitHub repository

The complete code of this project is available on GitHub:

https://github.com/FinianLandes/MA_Diffusion

The repository also includes a selection of generated samples using different models and parameters, as well as some training and test samples and a setup guide if you want to try out the code for yourself.

For sake of clarity, only the main Jupyter notebooks and the diffusion and the U-Net implementation are included here. All other files, such as the utils library are provided in the GitHub repository. Older models and additional methods can be found in earlier commits of the repository.

11.2 Hugging Face

A trained version of direct wave diffusion v6 can be found on Hugging Face.

https://huggingface.co/Chaerne/DirectWaveDiffusion_v6/blob/main/WaveDiffusion_v6.pth.